程序设计原理

继承与多态

任课教师： 喻　梅
于瑞国
王建荣
李雪威
赵满坤

review

Useful Array Functions:

```
#include <algorithm>
// [first, last)
```

min_element, max_element, sort, random_shuffle, and find

Dynamic Persistent Memory Allocation:

the new and delete operator

```
int* pointer = new int(8);
int* array = new int[10];
delete pointer;
delete [] array;
```

```
string* p = new string("abcdefg");
```

this Pointer:

Destructors:

```
class Circle{
public:
    …
    ~Circle();
    …
}
```

Copy Constructors:

```
Circle(const Circle&);
```

Objectives

- To define a derived class from a base class through inheritance.
- To enable generic programming by passing objects of a derived type to a parameter of a base class type.
- To know how to invoke the base class's constructors with arguments.
- To understand constructor and destructor chaining.
- To redefine functions in the derived class.
- To define generic functions using polymorphism.
- To enable dynamic binding using virtual functions.
- To understand dynamic binding.
- To access protected members of a base class from derived classes.
- To define abstract classes with pure virtual functions.

outline

- Base Classes and Derived Classes
- Constructors and Destructors
- Redefining Functions and Polymorphism
- Virtual functions and Dynamic Binding
- Abstract Classes and Pure Virtual Functions

1. Base Classes and Derived Classes
基类和派生类

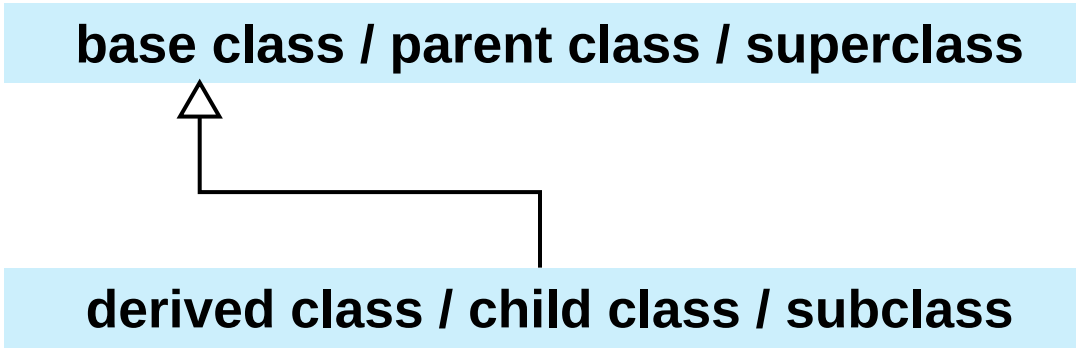
introduction

The three pillars of object-oriented programming are
Encapsulation (封装),
Inheritance (继承),
and **polymorphism (多态)**.

Base Classes and Derived Classes

Inheritance enables you to define a general class (i.e., a ***base class***) and later extend it to more specialized classes (i.e., ***derived classes***). The specialized classes inherit properties and functions from the general class.

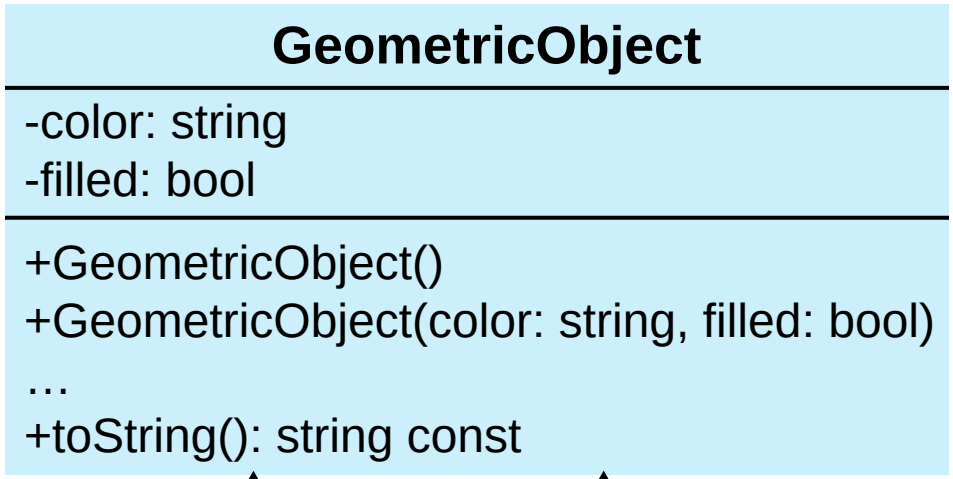
A derived class inherits ***accessible data fields*** and ***functions*** from its base class and may also add new data fields and functions.



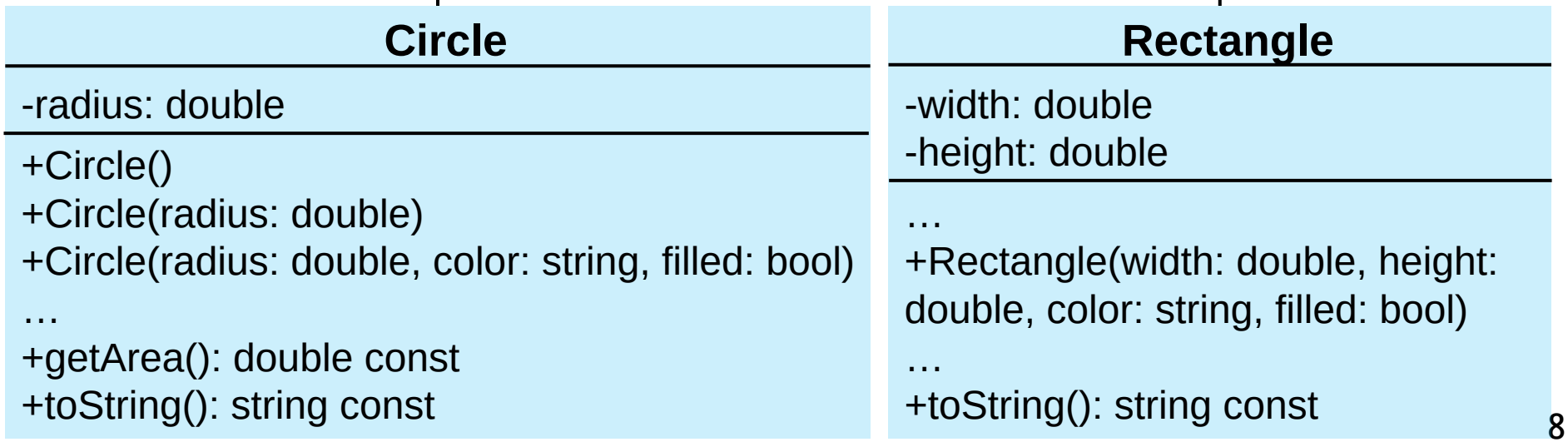
Base Classes and Derived Classes

UML: A triangle pointing to the base class is used to denote the inheritance relationship between the two classes involved.

base class:

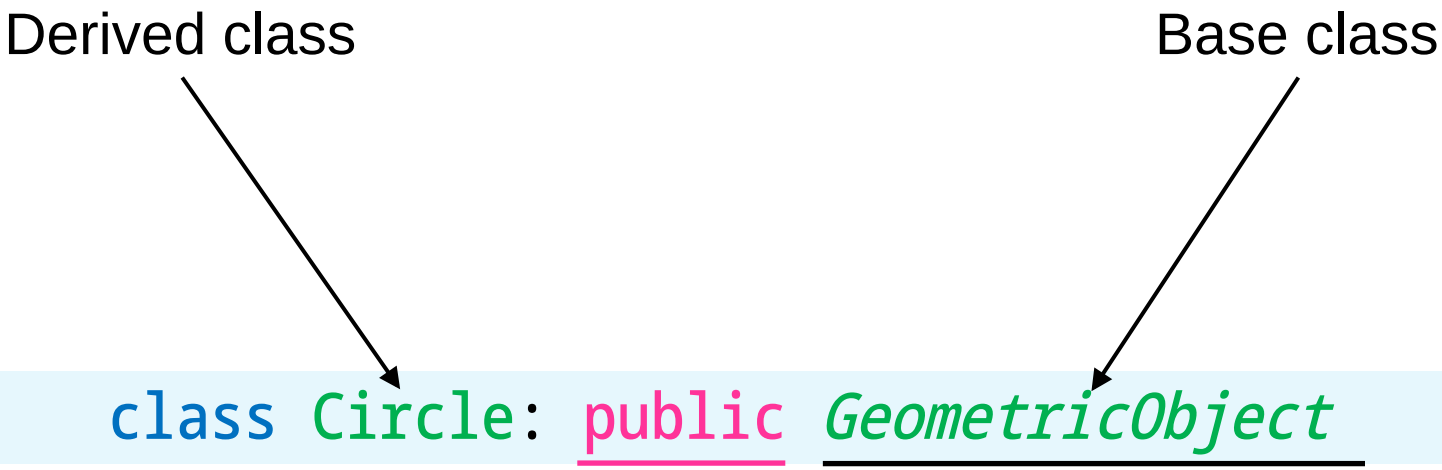


derived classes:



Base Classes and Derived Classes

The class definition for Circle defines that the Circle class is derived from the base class GeometricObject. The **syntax:**



public keyword: all public members in GeometricObject are inherited as public members in Circle.

The protected Keyword

A **protected** member of a class can be accessed from a derived ***class***.

The keywords **private**, **protected**, and **public** are known as visibility or accessibility keywords because they specify how class and class members are accessed. Their visibility increases in this order:

Visibility increases →

private, protected, public

```
class B
{
public:
  int i;

protected:
  int j;

private:
  int k;
};
```

```
class A: public B
{
public:
  void display() const
  {
    cout << i << endl; // Fine
    cout << j << endl; // Fine
    cout << k << endl; // Wrong
  }
};
```

```
int main()
{
  A a;
  cout << a.i << endl; // Fine
  cout << a.j << endl; // Wrong
  cout << a.k << endl; // Wrong

  return 0;
}
```

Base Classes and Derived Classes

Class A is derived from class B	public members in class B are inherited as	protected members in class B are inherited as	private members in class B are inherited as
`class A:public B`	`public`	`protected`	`private`
`class A:protected B`	`protected`	`protected`	`private`
`class A:private B`	`private`	`private`	`private`

Generic Programming

An object of a derived class can be passed wherever an object of a base type parameter is required. Thus a function can be used generically for a wide range of object arguments. This is known as generic programming.

Passing a Circle object or a Rectangle object to the GeometricObject parameter type in the displayGeometricObject function.

Base type parameter

```
void displayGeometricObject(const GeometricObject& shape) {
    cout << shape.getColor() << endl;
}
```

Base class object

```
int main(){
    displayGeometricObject(GeometricObject("black", true));
    displayGeometricObject(Circle(5));
    displayGeometricObject(Rectangle(2, 3));
    return 0;
}
```

Derived class object

2. Constructors and Destructors

构造函数和析构函数

Destructors

Every class has a destructor, which is called automatically when an object is deleted.

Every class has a default destructor if the destructor is not explicitly defined. Also, we could implement destructors to perform customized operations.

Destructors are named the same as constructors, but you must put a tilde character (~) in front:

Circle.h

```
class Circle{
public:
    Circle();
    Circle(double r);
    ~Circle();
    …
}
```

destructor

Circle.cpp

```
Circle::~Circle(){  …  }
```

Constructors and Destructors

The constructor of a derived class first calls its base class's constructor before it executes its own code.
The destructor of a derived class executes its own code then automatically calls its base class's destructor.

You can invoke the base class's constructor ***from the constructor initializer list*** of the derived class. The syntax is as follows:

- Invoking the no-arg constructor of its base class

```
DerivedClass(parameterList): BaseClass()
{
    // Perform initialization
}
```

- Invoking the base class constructor with the specified arguments

```
DerivedClass(parameterList): BaseClass(argumentList)
{
    // Perform initialization
}
```

Constructors and Destructors

- If a base constructor is not invoked explicitly, the base class's no-arg constructor is invoked by default.

```
Circle()
{
    radius = 1;
}
```

is equivalent to

```
Circle(): GeometricObject()
{
    radius = 1;
}
```

- The Circle(double radius, const string& color, bool filled) constructor can also be implemented by invoking the base class's constructor GeometricObject(const string& color, bool filled) as follows:

```
Circle::Circle(double radius, const string& color, bool filled)
: GeometricObject(color, filled), radius(radius)
{
}
```

Constructors and Destructors

```
class B : public A{…};
```

Constructor chaining

```
B()
{
  cout <<"Constructor B"<<endl;
}
```

```
A()
{
  cout <<"Constructor A"<<endl;
}
```

Destructor chaining

```
~B()
{
  cout <<"Destructor B"<<endl;
}
```

```
~A()
{
  cout <<"Destructor A"<<endl;
}
```

Constructors and Destructors

Constructor and Destructor chaining

```
class A {
public:
    A(){cout << "Constructor A" << endl;}
    ~A(){cout << "Destructor  A" << endl;}
};

class B : public A{
public:
    B(){cout << "  Constructor B" << endl;}
    ~B(){cout << "  Destructor  B" << endl;}
};
```

```
int main() {
    B b;
    return 0;
}
```

```
Constructor A
  Constructor B
  Destructor  B
Destructor  A
```

3. Redefining Functions and Polymorphism

函数重定义和多态

Redefining Functions

A function defined in the base class can be redefined in the derived classes.

GeometricObject.cpp

```
string GeometricObject::toString() const{
    return "Geometric Object";
}
```

Circle.cpp

```
string Circle::toString() const{
    return "Circle";
}
```

Rectangle.cpp

```
string Rectangle::toString() const{
    return "Rectangle";
}
```

```
GeometricObject g;
cout << g.toString() << endl;
Circle circle(5, "red", true);
cout << circle.toString() << endl;
Rectangle rect;
cout << rect.toString() << endl;
```

```
circle.GeometricObject::toString()
```

::
scope resolution operator
作用域解析运算符

Polymorphism

Polymorphism means that a variable of a supertype can refer to a subtype object.

The function displayGeometricObject takes a parameter of the GeometricObject type.

```
void displayGeometricObject(const GeometricObject& shape) {
    cout << shape.getColor() << endl;
}
```

You can invoke displayGeometricObject by passing any instance of GeometricObject, Circle, and Rectangle.

```
int main(){
    displayGeometricObject(GeometricObject("black", true));
    displayGeometricObject(Circle(5));
    displayGeometricObject(Rectangle(2, 3));
    return 0;
}
```

An object of a derived class can be used wherever its base class object is used.

4. Virtual functions and Dynamic Binding

虚函数和动态绑定

Virtual Functions and Dynamic Binding

A function can be implemented in several classes along the inheritance chain. ***Virtual functions*** enable the system to decide which function is invoked at runtime based on the ***actual type*** of the object .

```
void displayGeometricObject(const GeometricObject& shape) {
    cout << shape.toString() << endl;
}
```

```
Geometric Object
Geometric Object
Geometric Object
```

→

```
Geometric Object
Circle
Rectangle
```

Define the toString() function as virtual ***in the base class***:

```
virtual string toString() const;
```

Dynamic binding: C++ dynamically binds the implementation of the function at runtime, decided by the actual class of the object referenced by the variable.

Virtual Functions and Dynamic Binding

To enable dynamic binding for a function, you need to do two things:

1. The function must be defined ***virtual*** in the base class.
2. The variable that references the object must be passed ***by reference*** or passed ***as a pointer*** in the virtual function.

```
void displayGeometricObject(const GeometricObject& shape) {
    cout << shape.toString() << endl;
}
```

```
void displayGeometricObject(const GeometricObject* shape) {
    cout << (*shape).toString() << endl;
}
```

5. Abstract Classes and Pure Virtual Functions

抽象类和纯虚函数

Abstract Classes and Pure Virtual Functions

An abstract class CANNOT be used to **create objects**. An abstract class contains abstract functions, which are implemented in concrete derived classes.

abstract functions ═══ pure virtual functions

A pure virtual function is defined this way:

Place ***optional*** const for constant function here.

Indicates pure virtual function

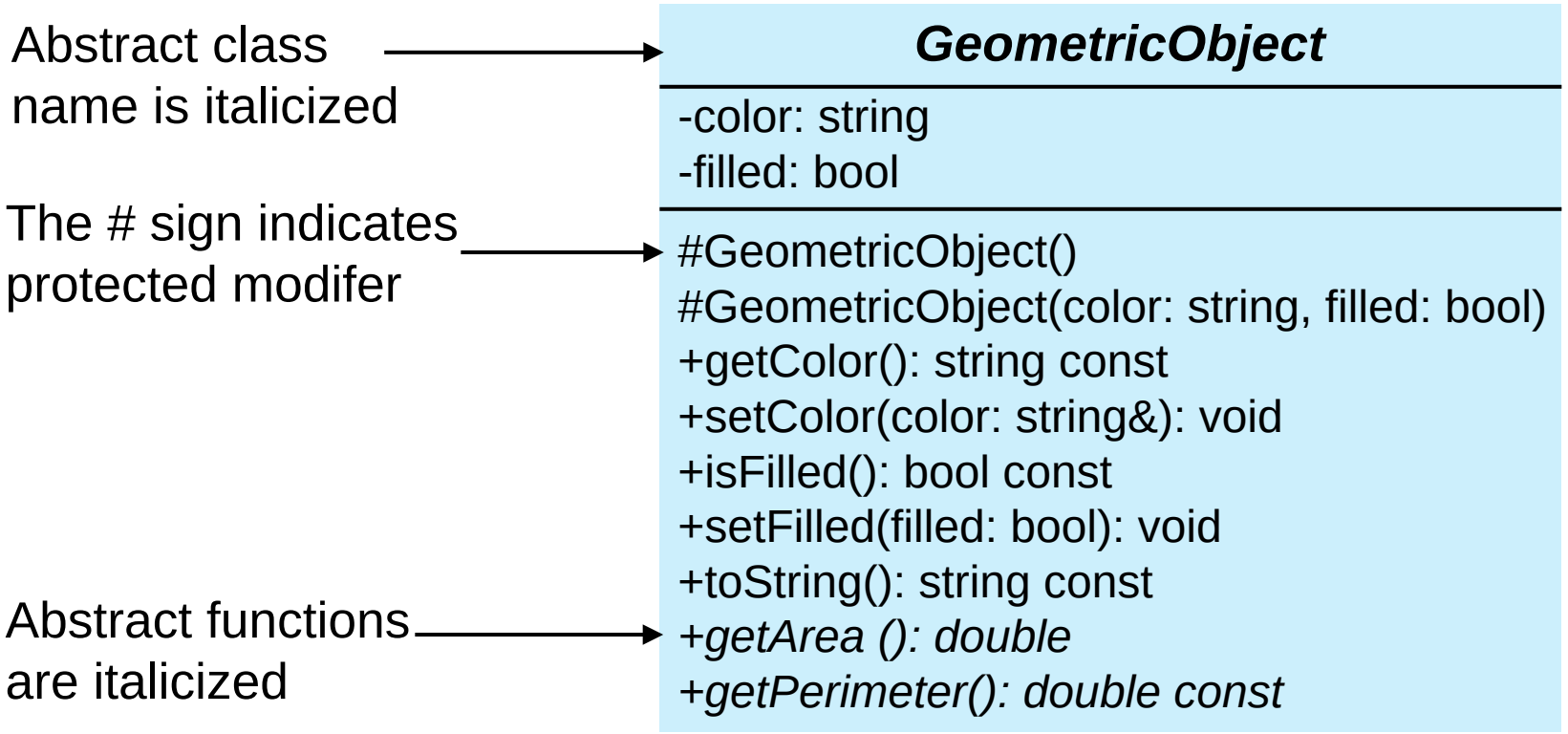
```
virtual double getArea() const = 0;
```

Note that a pure virtual function does not have a body or implementation in the base class.

Abstract Classes and Pure Virtual Functions

GeometricObject is just like a regular class, except that you cannot create objects from it because it is an abstract class.

UML:



Abstract Classes and Pure Virtual Functions

This example presents a program that creates two geometric objects (a circle and a rectangle), invokes the equalArea function to check whether the two objects have equal areas:

```
bool equalArea(GeometricObject& g1, GeometricObject& g2) {
    cout << g1.getArea()<< endl;
    cout << g2.getArea()<< endl;
    return g1.getArea() == g2.getArea();
}

int main() {
    Circle circle (5, "red", true);
    Rectangle rect;
    bool isEqual = equalArea(circle, rect);
    return 0;
}
```

GeometricObject.h
```
virtual double getArea() = 0;
```

Circle.h
```
double getArea(){return 3.14 * radius * radius;}
```

Rectangle.h
```
double getArea(){return width * height;}
```